

SQL ANALYSIS REPORT

E-Commerce Sales Dataset — Day 3

7-Day Data Analyst Portfolio Roadmap — Full Step-by-Step Documentation

By : Alya Shandy

1. Introduction

This report documents the data analysis process on the e-commerce sales dataset that was cleaned on Day 1 (cleaned_sales_data.csv, 3,933 rows). The analysis was carried out entirely using SQL (MySQL Server & MySQL Workbench), following the Day 3 checklist in the 7-day Data Analyst portfolio roadmap.

Like the Day 2 report, this one is written as a step-by-step tutorial — each section explains HOW to do something in MySQL Workbench (which menu to click, which wizard option to check, which query to type), not just the end result. The goal is so the process can be repeated or taught to someone else without having to recall everything from scratch.

Working file: business_analysis.sql — containing 15 business queries run against the sales_data table in the ecommerce_sales database, imported from cleaned_sales_data.csv.

2. Executive Summary

Table 1 summarizes the key KPIs from the SQL analysis.

Metric	Value
Total Revenue (Completed)	\$252,227.71
Total Orders (Completed)	3,220
Average Order Value	\$78.33
Average Revenue per Customer	\$288.59
East Region Contribution	74.4% of total revenue
Electronics Category Contribution	25.33% of total revenue

Table 1. Key Sales Performance Summary KPIs

3. Analysis Stages (Tutorial)

3.1 Installing MySQL Server & Workbench

Before any query can be run, MySQL Server and MySQL Workbench need to be installed first.

1. Download the MySQL Installer for Windows from the official MySQL website, choosing the full version (around 565.9 MB) which already includes MySQL Server and MySQL Workbench together.

2. Run the installer, then on the setup type selection choose "Full" so that all components (server, workbench, connector) are installed at once.
3. Continue through the built-in installation wizard — additional options that aren't needed don't have to be checked.
4. Create a root password for MySQL Server during the account configuration step. This password is used every time you connect to the database.
5. Complete the installation until you click "Finish", then open MySQL Workbench and create a new connection using the root password you just created.

3.2 Creating the Table & Importing Data

1. Create a new database (schema) named `ecommerce_sales`, then create a `sales_data` table with columns matching the dataset structure.
2. Right-click the `sales_data` table in the left panel → select "Table Data Import Wizard".
3. Click "Browse...", select the `cleaned_sales_data.csv` file, then click Next.
4. Make sure "Use existing table" is automatically set to `ecommerce_sales.sales_data`, then check "Truncate table before import" so the table is emptied before being filled — useful for preventing duplicate data if the import is run again.
5. Click Next then Finish to run the import, then verify the number of rows that were loaded.

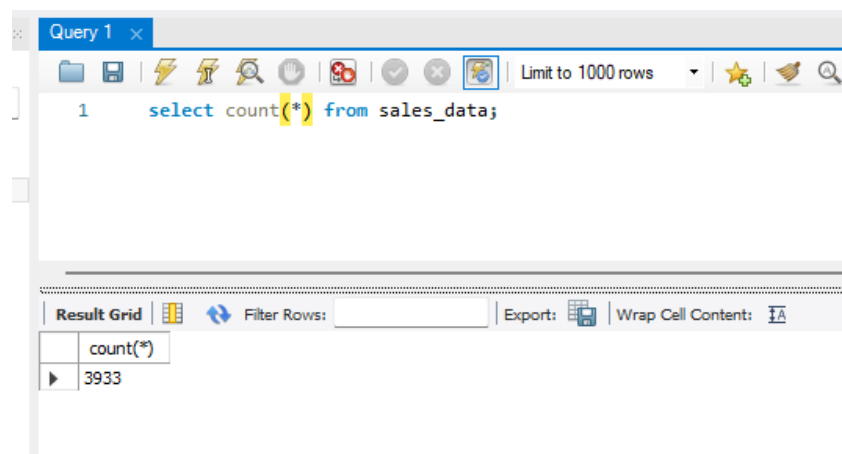


Figure 1. Verifying the number of imported rows — 3,933 rows, matching exactly the Day 1 cleaning result

The `order_date` column is stored as `varchar` (not `DATE`), so any time a time-based analysis (monthly, daily) is needed, this column must first be converted using the `STR_TO_DATE()` function.

Query 1 x

Limit to 1000 rows

```
1 describe sales_data;
2
```

Result Grid

Filter Rows: Export: Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
▶	order_id	varchar(15)	YES		NULL	
	order_date	varchar(10)	YES		NULL	
	customer_id	varchar(15)	YES		NULL	
	customer_name	varchar(100)	YES		NULL	
	region	varchar(20)	YES		NULL	
	category	varchar(30)	YES		NULL	
	product_name	varchar(50)	YES		NULL	
	quantity	int	YES		NULL	
	unit_price	decimal(10,2)	YES		NULL	
	order_status	varchar(20)	YES		NULL	
	payment_method	varchar(20)	YES		NULL	
	revenue	decimal(10,2)	YES		NULL	

Figure 2. Table structure (DESCRIBE sales_data) — 12 columns with their respective data types

1 • select * from sales_data limit 5;

2

Result Grid

Filter Rows: Export: Wrap Cell Content: Fetch rows:

	order_id	order_date	customer_id	customer_name	region	category	product_name	quantity	unit_price	order_status	payment_method	revenue
▶	ORD100363	6/17/2023	CUST1454	Fajar Wijaya	East	Beauty	Lip Balm	1	6.00	Completed	Gift Card	6.00
	ORD101186	10/17/2024	CUST1674	Dewi Lee	East	Beauty	Lip Balm	1	6.22	Completed	Gift Card	6.22
	ORD101364	7/11/2024	CUST1201	Linda Johnson	East	Beauty	Shampoo	2	6.22	Completed	Credit Card	12.44
	ORD102845	11/20/2023	CUST1361	Rina Brown	East	Beauty	Moisturizer	2	6.27	Completed	Debit Card	12.54
	ORD102156	1/11/2024	CUST1798	Siti Anderson	East	Beauty	Lip Balm	1	6.38	Completed	Gift Card	6.38

Figure 3. Sample of 5 rows (SELECT * LIMIT 5) — column structure matching the Day 1 cleaning result

3.3 Query 1 — Total Revenue

```
SELECT SUM(revenue) AS total_revenue
FROM sales_data
WHERE order_status = 'Completed';
```

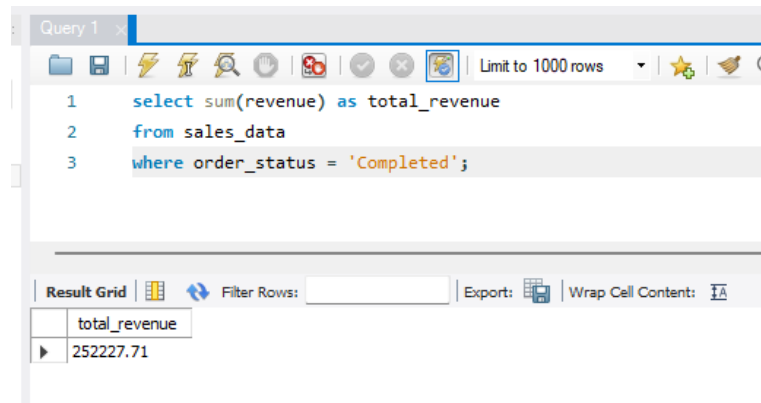


Figure 4. Query 1 result — total_revenue = 252,227.71

The filter WHERE order_status = 'Completed' is used because cancelled/returned orders aren't counted as real business revenue.

3.4 Query 2 — Total Orders

```

SELECT COUNT(*) AS total_orders
FROM sales_data
WHERE order_status = 'Completed';

```

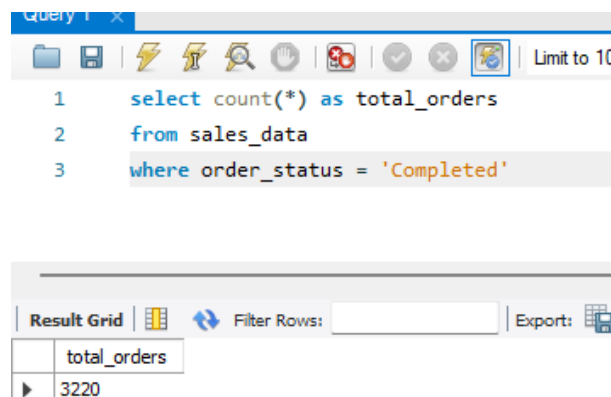


Figure 5. Query 2 result — total_orders = 3,220

3.5 Query 3 — Average Order Value (AOV)

```

SELECT ROUND(SUM(revenue) / COUNT(*), 2) AS avg_order_value
FROM sales_data
WHERE order_status = 'Completed';

```

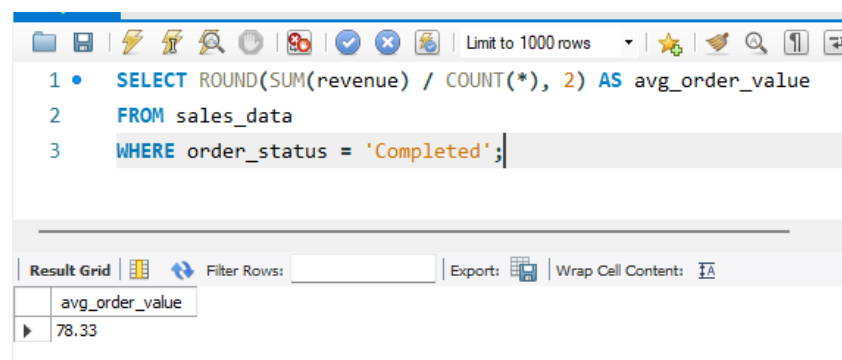
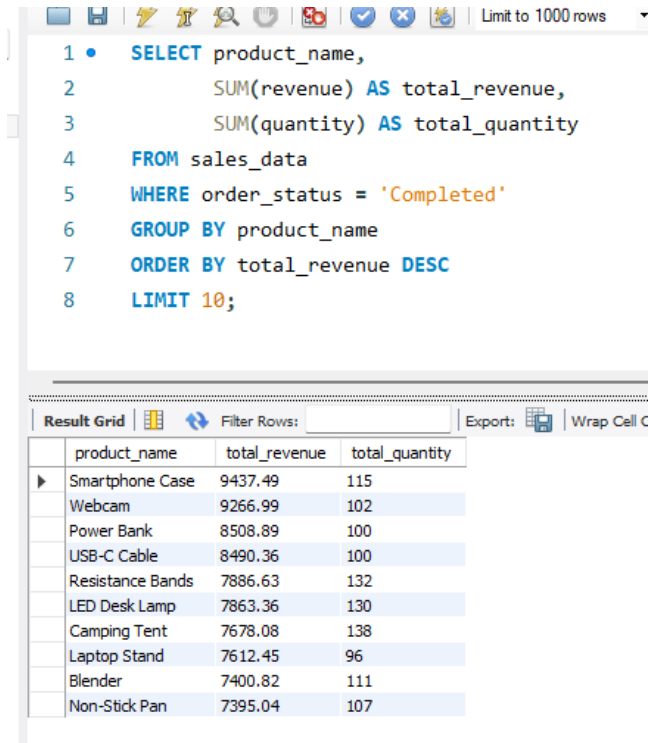


Figure 6. Query 3 result — avg_order_value = 78.33

3.6 Query 4 — Top 10 Products by Revenue

```
SELECT product_name,
       SUM(revenue) AS total_revenue,
       SUM(quantity) AS total_quantity
FROM sales_data
WHERE order_status = 'Completed'
GROUP BY product_name
ORDER BY total_revenue DESC
LIMIT 10;
```



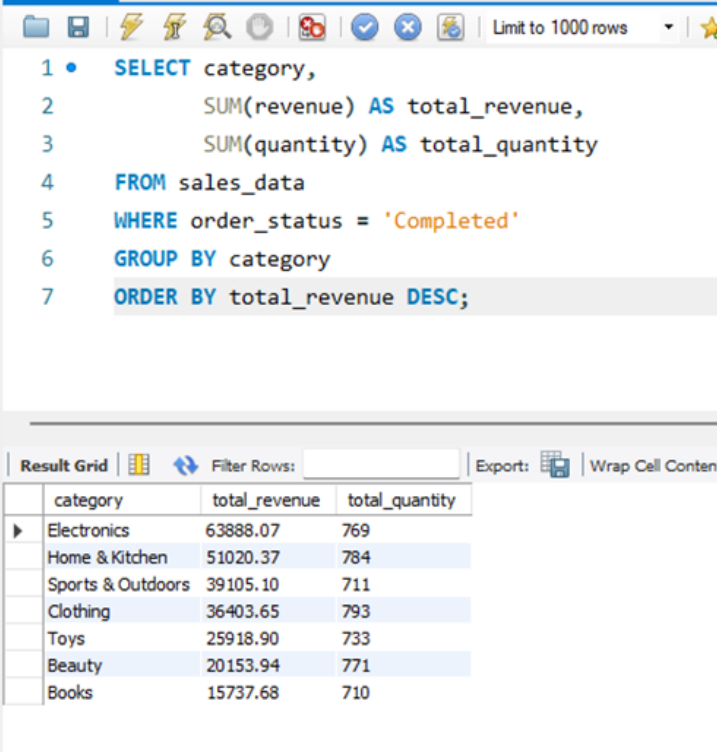
The screenshot shows a SQL query editor with the query from the previous block. Below the editor is a 'Result Grid' showing the results of the query. The table has three columns: product_name, total_revenue, and total_quantity. The results are ordered by total_revenue in descending order.

product_name	total_revenue	total_quantity
Smartphone Case	9437.49	115
Webcam	9266.99	102
Power Bank	8508.89	100
USB-C Cable	8490.36	100
Resistance Bands	7886.63	132
LED Desk Lamp	7863.36	130
Camping Tent	7678.08	138
Laptop Stand	7612.45	96
Blender	7400.82	111
Non-Stick Pan	7395.04	107

Figure 7. Query 4 result — Smartphone Case leads with 9,437.49 in revenue

3.7 Query 5 — Top Categories

```
SELECT category,
       SUM(revenue) AS total_revenue,
       SUM(quantity) AS total_quantity
FROM sales_data
WHERE order_status = 'Completed'
GROUP BY category
ORDER BY total_revenue DESC;
```



```

1 • SELECT category,
2       SUM(revenue) AS total_revenue,
3       SUM(quantity) AS total_quantity
4 FROM sales_data
5 WHERE order_status = 'Completed'
6 GROUP BY category
7 ORDER BY total_revenue DESC;

```

category	total_revenue	total_quantity
Electronics	63888.07	769
Home & Kitchen	51020.37	784
Sports & Outdoors	39105.10	711
Clothing	36403.65	793
Toys	25918.90	733
Beauty	20153.94	771
Books	15737.68	710

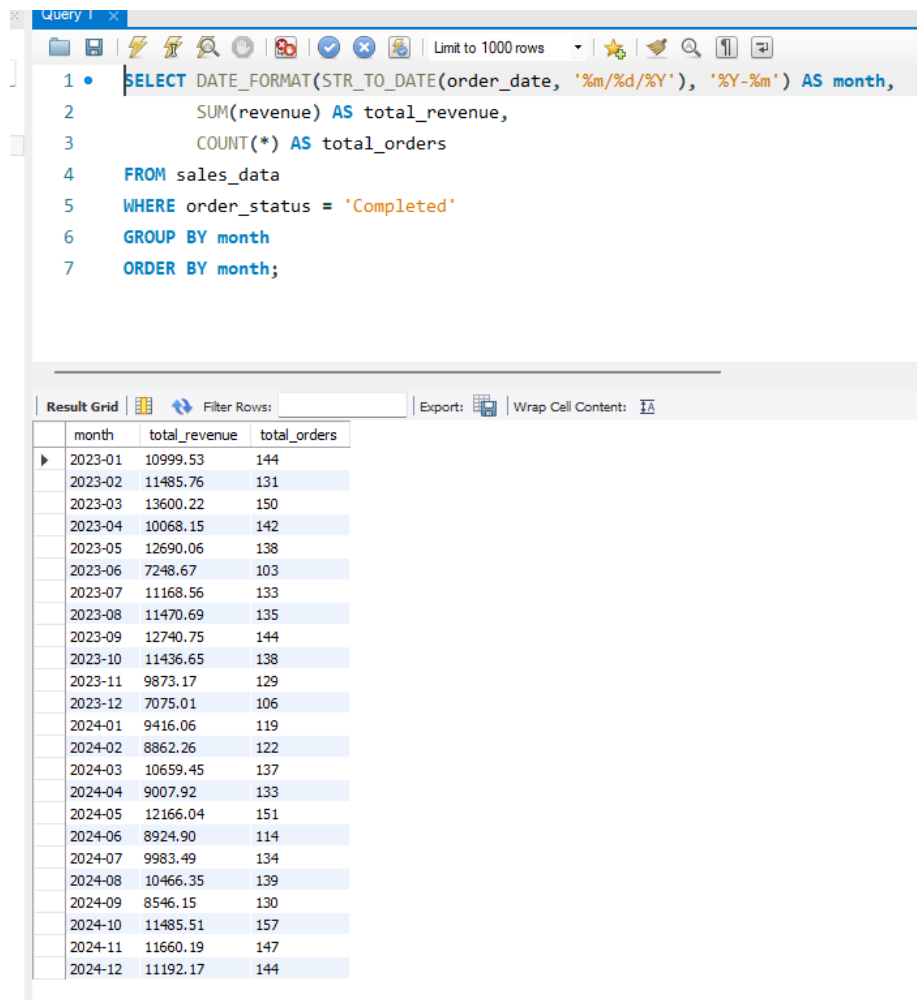
Figure 8. Query 5 result — Electronics leads with 63,888.07 in revenue

3.8 Query 6 — Monthly Sales

```

SELECT DATE_FORMAT(STR_TO_DATE(order_date, '%m/%d/%Y'), '%Y-%m') AS month,
       SUM(revenue) AS total_revenue,
       COUNT(*) AS total_orders
FROM sales_data
WHERE order_status = 'Completed'
GROUP BY month
ORDER BY month;

```



Query 6 SQL:

```

1 • SELECT DATE_FORMAT(STR_TO_DATE(order_date, '%m/%d/%Y'), '%Y-%m') AS month,
2       SUM(revenue) AS total_revenue,
3       COUNT(*) AS total_orders
4 FROM sales_data
5 WHERE order_status = 'Completed'
6 GROUP BY month
7 ORDER BY month;

```

Result Grid:

month	total_revenue	total_orders
2023-01	10999.53	144
2023-02	11485.76	131
2023-03	13600.22	150
2023-04	10068.15	142
2023-05	12690.06	138
2023-06	7248.67	103
2023-07	11168.56	133
2023-08	11470.69	135
2023-09	12740.75	144
2023-10	11436.65	138
2023-11	9873.17	129
2023-12	7075.01	106
2024-01	9416.06	119
2024-02	8862.26	122
2024-03	10659.45	137
2024-04	9007.92	133
2024-05	12166.04	151
2024-06	8924.90	114
2024-07	9983.49	134
2024-08	10466.35	139
2024-09	8546.15	130
2024-10	11485.51	157
2024-11	11660.19	147
2024-12	11192.17	144

Figure 9. Query 6 result — monthly revenue from Jan 2023 to Dec 2024 (24 months)

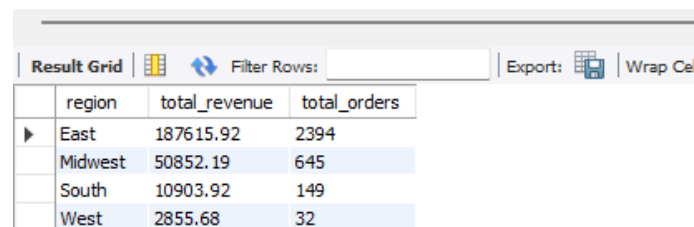
3.9 Query 7 — Regional Sales

```

SELECT region,
       SUM(revenue) AS total_revenue,
       COUNT(*) AS total_orders
FROM sales_data
WHERE order_status = 'Completed'
GROUP BY region
ORDER BY total_revenue DESC;

```

```
1 • SELECT region,  
2       SUM(revenue) AS total_revenue,  
3       COUNT(*) AS total_orders  
4 FROM sales_data  
5 WHERE order_status = 'Completed'  
6 GROUP BY region  
7 ORDER BY total_revenue DESC;
```



	region	total_revenue	total_orders
►	East	187615.92	2394
	Midwest	50852.19	645
	South	10903.92	149
	West	2855.68	32

Figure 10. Query 7 result — the East region dominates (187,615.92 out of 252,227.71 total)

3.10 Query 8 — Repeat Customers

```
SELECT customer_id,
       customer_name,
       COUNT(*) AS total_orders
FROM sales_data
WHERE order_status = 'Completed'
GROUP BY customer_id, customer_name
HAVING COUNT(*) > 1
ORDER BY total_orders DESC;
```

1 • SELECT customer_id,
2 customer_name,
3 COUNT(*) AS total_orders
4 FROM sales_data
5 WHERE order_status = 'Completed'
6 GROUP BY customer_id, customer_name
7 HAVING COUNT(*) > 1
8 ORDER BY total_orders DESC;

Result Grid | Filter Rows: | Exports: | Wrap Cell Co

	customer_id	customer_name	total_orders
▶	CUST1543	Ayu Wilson	11
	CUST1145	Linda Smith	11
	CUST1755	Budi Sari	10
	CUST1382	David Hidayat	10
	CUST1674	Dewi Lee	9
	CUST1798	Siti Anderson	9
	CUST1839	John Tan	9
	CUST1745	Ahmad Halim	9
	CUST1865	Ayu Salim	9
	CUST1514	Rina Wijaya	9
	CUST1834	Andi Nugroho	9
	CUST1655	John Tan	8
	CUST1412	Siti Anderson	8
	CUST1783	Ayu Taylor	8
	CUST1436	Lisa Garcia	8
	CUST1052	Fajar Halim	8
	CUST1248	Andi Pratama	8
	CUST1833	Putri Yusuf	8
	CUST1611	Maria Sari	8
	CUST1451	Ahmad Garcia	8
	CUST1808	Michael Kurnia...	8
	CUST1222	Dewi Sari	8
	CUST1779	Andi Yusuf	8
	CUST1223	Michael Halim	7
	CUST1060	Ahmad Wilson	7
	CUST1818	Siti Salim	7
	CUST1738	Fajar Halim	7
	CUST1679	Siti Halim	7
	CUST1761	Ahmad Johnson	7
	CUST1270	Andi Salim	7
	CUST1734	Ahmad Wilson	7
	CUST1726	John Tan	7
	CUST1005	Dewi Wijaya	7
	CUST1095	Sarah Smith	7
	CUST1596	Ayu Garcia	7
	CUST1240	Linda Chen	7
	CUST1460	Rina Sari	7
	CUST1649	Rina Wilson	7
	CUST1172	Nina Lee	7
	CUST1041	Michael Wijaya	7
	CUST1499	David Santos	7

Result 11 x

Figure 11. Query 8 result — Ayu Wilson and Linda Smith order most frequently (11x)

3.11 Query 9 — Highest-Value Customers

```
SELECT customer_id,
       customer_name,
       SUM(revenue) AS total_spent,
       COUNT(*) AS total_orders
FROM sales_data
```

```

WHERE order_status = 'Completed'
GROUP BY customer_id, customer_name
ORDER BY total_spent DESC
LIMIT 10;

```

Query 1 x

Limit to 1000 rows

```

1 • SELECT customer_id,
2         customer_name,
3         SUM(revenue) AS total_spent,
4         COUNT(*) AS total_orders
5 FROM sales_data
6 WHERE order_status = 'Completed'
7 GROUP BY customer_id, customer_name
8 ORDER BY total_spent DESC
9 LIMIT 10;

```

Result Grid | Filter Rows: | Export: | Wrap Cell C

	customer_id	customer_name	total_spent	total_orders
▶	CUST1644	Andi Salim	1285.26	6
	CUST1054	Putri Garcia	1054.93	3
	CUST1543	Ayu Wilson	972.76	11
	CUST1755	Budi Sari	966.12	10
	CUST1248	Andi Pratama	951.12	8
	CUST1465	Nina Wilson	944.29	4
	CUST1819	Lisa Salim	938.90	4
	CUST1579	Kevin Tan	936.35	6
	CUST1870	Dewi Chen	906.64	6
	CUST1836	Kevin Salim	904.89	5

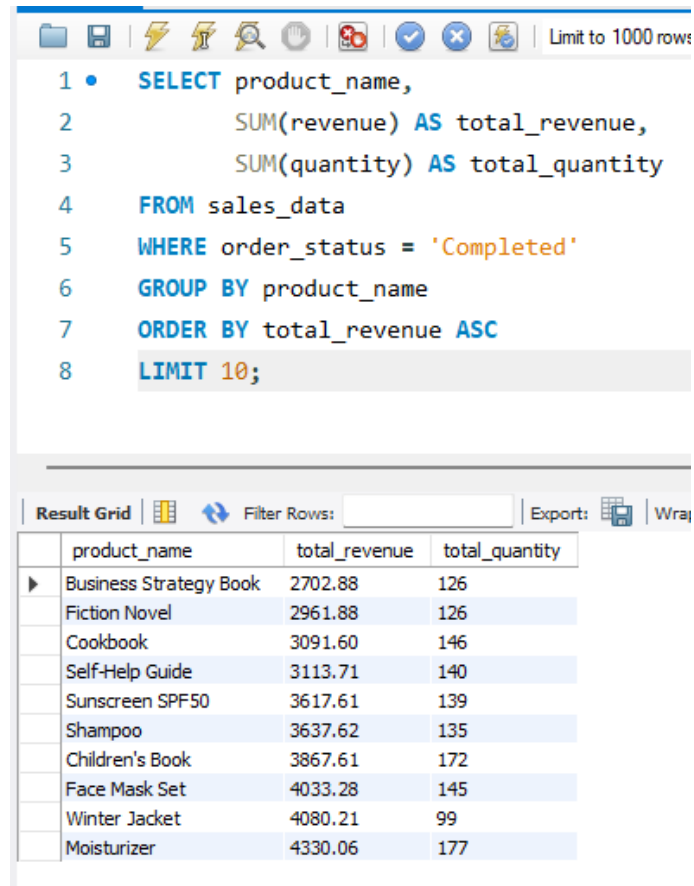
Figure 12. Query 9 result — Andi Salim has the highest total spend (1,285.26) from just 6 orders

3.12 Query 10 — Lowest-Performing Products

```

SELECT product_name,
       SUM(revenue) AS total_revenue,
       SUM(quantity) AS total_quantity
FROM sales_data
WHERE order_status = 'Completed'
GROUP BY product_name
ORDER BY total_revenue ASC
LIMIT 10;

```



```

1 • SELECT product_name,
2         SUM(revenue) AS total_revenue,
3         SUM(quantity) AS total_quantity
4 FROM sales_data
5 WHERE order_status = 'Completed'
6 GROUP BY product_name
7 ORDER BY total_revenue ASC
8 LIMIT 10;

```

product_name	total_revenue	total_quantity
Business Strategy Book	2702.88	126
Fiction Novel	2961.88	126
Cookbook	3091.60	146
Self-Help Guide	3113.71	140
Sunscreen SPF50	3617.61	139
Shampoo	3637.62	135
Children's Book	3867.61	172
Face Mask Set	4033.28	145
Winter Jacket	4080.21	99
Moisturizer	4330.06	177

Figure 13. Query 10 result — Books-category products dominate the lowest-revenue list

3.13 Query 11 — Growth Month-over-Month (MoM)

```

SELECT month, total_revenue,
       ROUND(
         (total_revenue - LAG(total_revenue) OVER (ORDER BY month))
         / LAG(total_revenue) OVER (ORDER BY month) * 100, 2
       ) AS growth_pct
FROM (
  SELECT DATE_FORMAT(STR_TO_DATE(order_date, '%m/%d/%Y'), '%Y-%m') AS month,
         SUM(revenue) AS total_revenue
  FROM sales_data WHERE order_status = 'Completed' GROUP BY month
) AS monthly ORDER BY month;

```

```

1 • SELECT month,
2     total_revenue,
3     ROUND(
4         (total_revenue - LAG(total_revenue) OVER (ORDER BY month))
5         / LAG(total_revenue) OVER (ORDER BY month) * 100, 2
6     ) AS growth_pct
7 FROM (
8     SELECT DATE_FORMAT(STR_TO_DATE(order_date, '%m/%d/%Y'), '%Y-%m') AS month,
9         SUM(revenue) AS total_revenue
10    FROM sales_data
11   WHERE order_status = 'Completed'
12   GROUP BY month
13 ) AS monthly
14 ORDER BY month;

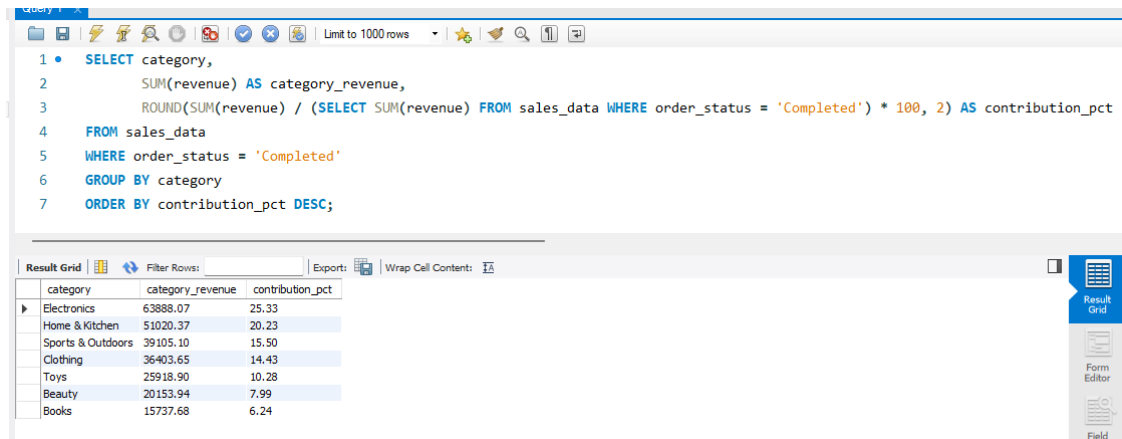
```

month	total_revenue	growth_pct
2023-01	10999.53	NULL
2023-02	11485.76	4.42
2023-03	13600.22	18.41
2023-04	10068.15	-25.97
2023-05	12690.06	26.04
2023-06	7248.67	-42.88
2023-07	11168.56	54.08
2023-08	11470.69	2.71
2023-09	12740.75	11.07
2023-10	11436.65	-10.24
2023-11	9873.17	-13.67
2023-12	7075.01	-28.34
2024-01	9416.06	33.09
2024-02	8862.26	-5.88
2024-03	10659.45	20.28
2024-04	9007.92	-15.49
2024-05	12166.04	35.06
2024-06	8924.90	-26.64
2024-07	9983.49	11.86
2024-08	10466.35	4.84
2024-09	8546.15	-18.35
2024-10	11485.51	34.39
2024-11	11660.19	1.52
2024-12	11192.17	-4.01

Figure 14. Query 11 result — the LAG() window function calculates month-over-month growth (-42.88% to +54.08%)

3.14 Query 12 — Category Contribution (%)

```
SELECT category, SUM(revenue) AS category_revenue,
       ROUND(SUM(revenue) / (SELECT SUM(revenue) FROM sales_data
                             WHERE order_status = 'Completed') * 100, 2) AS contribution_pct
FROM sales_data WHERE order_status = 'Completed'
GROUP BY category ORDER BY contribution_pct DESC;
```



The screenshot shows a SQL query editor with the following query:

```
1 • SELECT category,
2       SUM(revenue) AS category_revenue,
3       ROUND(SUM(revenue) / (SELECT SUM(revenue) FROM sales_data WHERE order_status = 'Completed') * 100, 2) AS contribution_pct
4 FROM sales_data
5 WHERE order_status = 'Completed'
6 GROUP BY category
7 ORDER BY contribution_pct DESC;
```

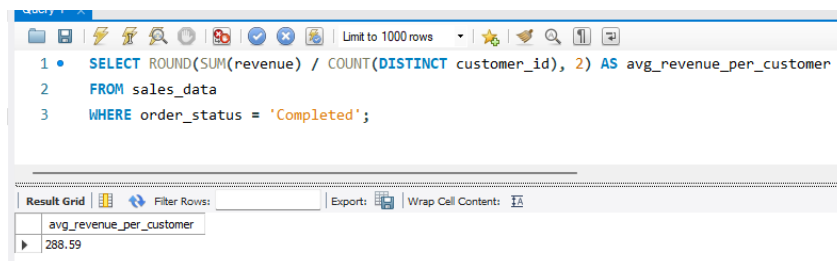
Below the query editor is a 'Result Grid' showing the results of the query:

category	category_revenue	contribution_pct
Electronics	63888.07	25.33
Home & Kitchen	51020.37	20.23
Sports & Outdoors	39105.10	15.50
Clothing	36403.65	14.43
Toys	25918.90	10.28
Beauty	20153.94	7.99
Books	15737.68	6.24

Figure 15. Query 12 result — Electronics contributes 25.33% of total revenue

3.15 Query 13 — Average Revenue per Customer

```
SELECT ROUND(SUM(revenue) / COUNT(DISTINCT customer_id), 2) AS avg_revenue_per_customer
FROM sales_data WHERE order_status = 'Completed';
```



The screenshot shows a SQL query editor with the following query:

```
1 • SELECT ROUND(SUM(revenue) / COUNT(DISTINCT customer_id), 2) AS avg_revenue_per_customer
2 FROM sales_data
3 WHERE order_status = 'Completed';
```

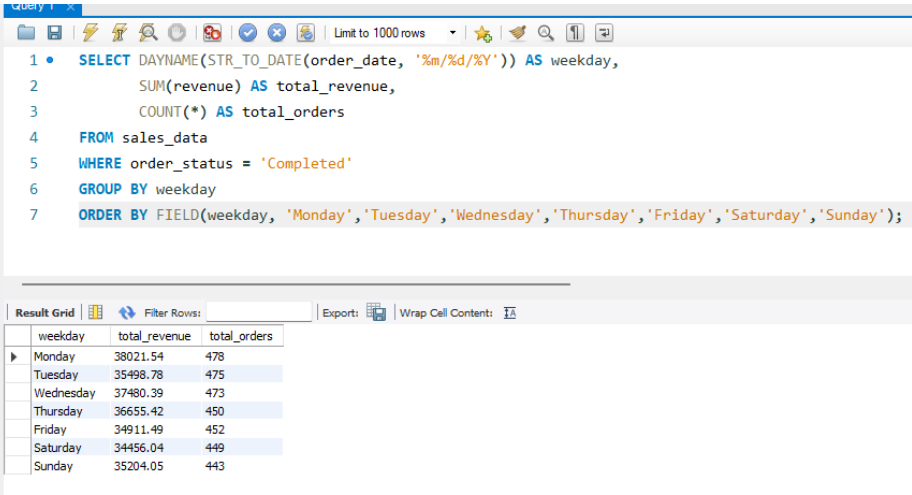
Below the query editor is a 'Result Grid' showing the results of the query:

avg_revenue_per_customer
288.59

Figure 16. Query 13 result — avg_revenue_per_customer = 288.59

3.16 Query 14 — Sales by Weekday

```
SELECT DAYNAME(STR_TO_DATE(order_date, '%m/%d/%Y')) AS weekday,
       SUM(revenue) AS total_revenue, COUNT(*) AS total_orders
FROM sales_data WHERE order_status = 'Completed'
GROUP BY weekday
ORDER BY
FIELD(weekday, 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday');
```



```

1 • SELECT DAYNAME(STR_TO_DATE(order_date, '%m/%d/%Y')) AS weekday,
2     SUM(revenue) AS total_revenue,
3     COUNT(*) AS total_orders
4 FROM sales_data
5 WHERE order_status = 'Completed'
6 GROUP BY weekday
7 ORDER BY FIELD(weekday, 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday');

```

weekday	total_revenue	total_orders
Monday	38021.54	478
Tuesday	35498.78	475
Wednesday	37480.39	473
Thursday	36655.42	450
Friday	34911.49	452
Saturday	34456.04	449
Sunday	35204.05	443

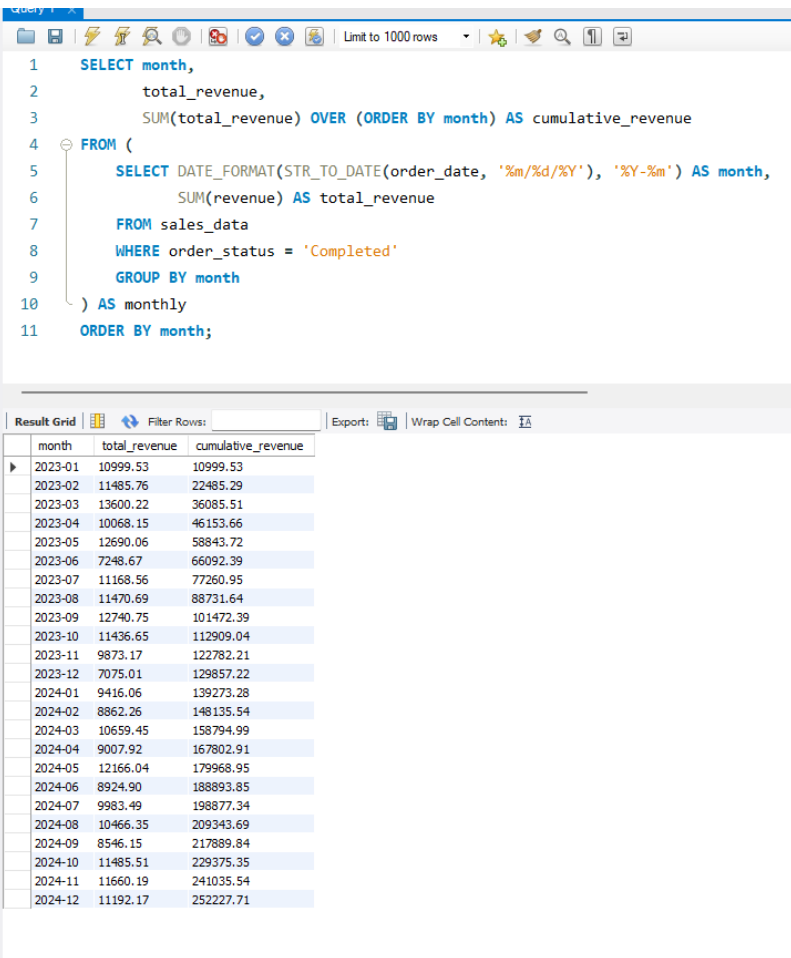
Figure 17. Query 14 result — sales are fairly even across the week, with Monday highest and Saturday lowest

3.17 Query 15 — Sales Trend (Cumulative Revenue)

```

SELECT month, total_revenue,
       SUM(total_revenue) OVER (ORDER BY month) AS cumulative_revenue
FROM (
  SELECT DATE_FORMAT(STR_TO_DATE(order_date, '%m/%d/%Y'), '%Y-%m') AS month,
         SUM(revenue) AS total_revenue
  FROM sales_data WHERE order_status = 'Completed' GROUP BY month
) AS monthly ORDER BY month;

```



```

1 SELECT month,
2        total_revenue,
3        SUM(total_revenue) OVER (ORDER BY month) AS cumulative_revenue
4 FROM (
5   SELECT DATE_FORMAT(STR_TO_DATE(order_date, '%m/%d/%Y'), '%Y-%m') AS month,
6          SUM(revenue) AS total_revenue
7   FROM sales_data
8   WHERE order_status = 'Completed'
9   GROUP BY month
10  ) AS monthly
11 ORDER BY month;

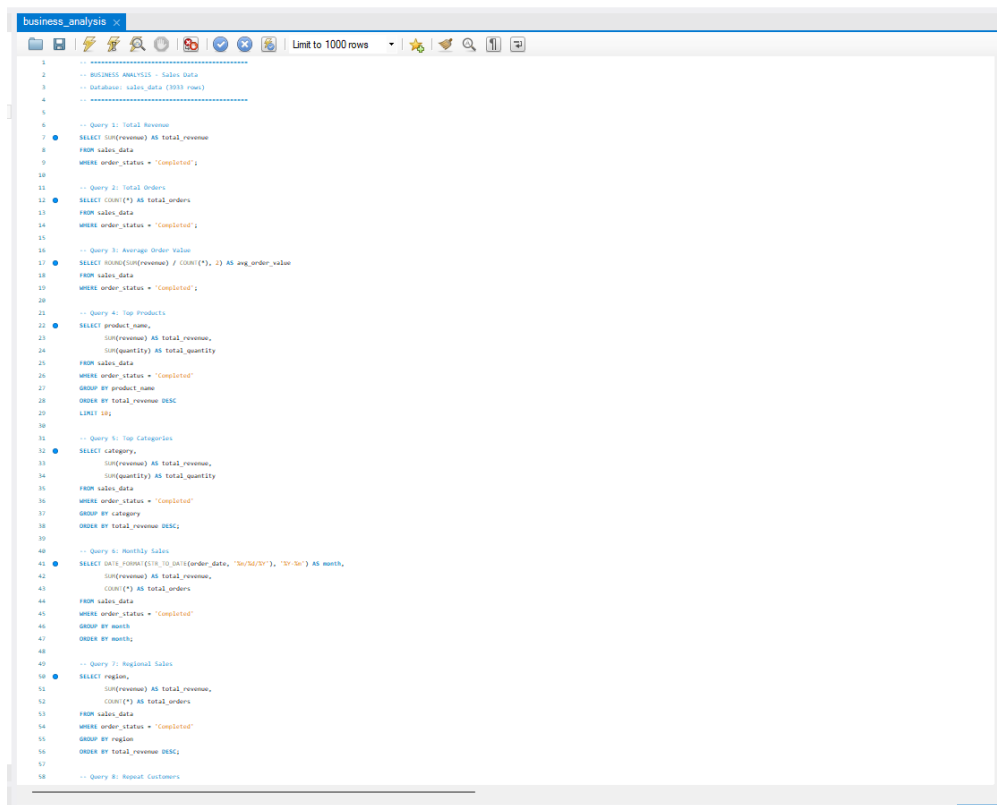
```

month	total_revenue	cumulative_revenue
2023-01	10999.53	10999.53
2023-02	11485.76	22485.29
2023-03	13600.22	36085.51
2023-04	10068.15	46153.66
2023-05	12690.06	58843.72
2023-06	7248.67	66092.39
2023-07	11168.56	77260.95
2023-08	11470.69	88731.64
2023-09	12740.75	101472.39
2023-10	11436.65	112909.04
2023-11	9873.17	122782.21
2023-12	7075.01	129857.22
2024-01	9416.06	139273.28
2024-02	8862.26	148135.54
2024-03	10659.45	158794.99
2024-04	9007.92	167802.91
2024-05	12166.04	179968.95
2024-06	8924.90	188893.85
2024-07	9983.49	198877.34
2024-08	10466.35	209343.69
2024-09	8546.15	217889.84
2024-10	11485.51	229375.35
2024-11	11660.19	241035.54
2024-12	11192.17	252227.71

Figure 18. Query 15 result — final cumulative_revenue (Dec 2024) = 252,227.71, consistent with Query 1

3.18 Saving the Queries — business_analysis.sql

All 15 queries were combined into a single file and saved via File → Save Script As in MySQL Workbench, so it can easily be reused or reviewed by others.



```

1  -- =====
2  -- BUSINESS ANALYSIS - Sales Data
3  -- Database: sales_data (3933 rows)
4  -- =====
5
6  -- Query 1: Total Revenue
7  SELECT SUM(revenue) AS total_revenue
8  FROM sales_data
9  WHERE order_status = 'Completed';
10
11 -- Query 2: Total Orders
12 SELECT COUNT(*) AS total_orders
13 FROM sales_data
14 WHERE order_status = 'Completed';
15
16 -- Query 3: Average Order Value
17 SELECT ROUND(SUM(revenue) / COUNT(*), 2) AS avg_order_value
18 FROM sales_data
19 WHERE order_status = 'Completed';
20
21 -- Query 4: Top Products
22 SELECT product_name,
23        SUM(revenue) AS total_revenue,
24        SUM(quantity) AS total_quantity
25 FROM sales_data
26 WHERE order_status = 'Completed'
27 GROUP BY product_name
28 ORDER BY total_revenue DESC
29 LIMIT 10;
30
31 -- Query 5: Top Categories
32 SELECT category,
33        SUM(revenue) AS total_revenue,
34        SUM(quantity) AS total_quantity
35 FROM sales_data
36 WHERE order_status = 'Completed'
37 GROUP BY category
38 ORDER BY total_revenue DESC;
39
40 -- Query 6: Monthly Sales
41 SELECT DATE_FORMAT(ORDER_DATE, '%Y-%m') AS month,
42        SUM(revenue) AS total_revenue,
43        COUNT(*) AS total_orders
44 FROM sales_data
45 WHERE order_status = 'Completed'
46 GROUP BY month
47 ORDER BY month;
48
49 -- Query 7: Regional Sales
50 SELECT region,
51        SUM(revenue) AS total_revenue,
52        COUNT(*) AS total_orders
53 FROM sales_data
54 WHERE order_status = 'Completed'
55 GROUP BY region
56 ORDER BY total_revenue DESC;
57
58 -- Query 8: Repeat Customers

```

Figure 19. The `business_analysis.sql` file saved with comments for each query

4. Business Insights

The eight insights below were compiled from all the figures and patterns found throughout the SQL analysis above.

1. Extreme Dependence on the East Region

The East region accounts for 74.4% of total revenue (187,615.92 out of 252,227.71) and 74.3% of total orders (2,394 out of 3,220), while West contributes only 1.1% (2,855.68, 32 orders). This extreme imbalance shows the business is highly dependent on a single region — a significant risk if East's performance were to decline.

2. Transaction Value Doesn't Always Track Order Frequency

The most frequent customer (Ayu Wilson, 11 orders) is not the customer who spent the most. Andi Salim actually recorded the highest total spend (1,285.26) from just 6 orders — indicating a segment of high-value customers distinct from the most loyal (most frequent) segment.

3. Electronics Is the Dominant Category and Product Line

Electronics is the highest-revenue category (63,888.07, 25.33% of total), followed by Home & Kitchen (51,020.37, 20.23%). Best-selling products (Smartphone Case, Webcam, Power Bank) also belong to the Electronics category — making it the primary sales driver.

4. Books Is the Weakest-Performing Category

Books is the lowest-revenue category (15,737.68, 6.24% of total), less than a quarter of Electronics' revenue. Individual products such as Business Strategy Book and Fiction Novel also rank among the lowest revenue products — this category could be re-evaluated in terms of product range or promotional strategy.

5. No Consistent Seasonal Pattern

Month-over-month growth fluctuates sharply with no clear seasonal pattern — ranging from -42.88% (June 2023) to +54.08% (July 2023). This volatility suggests revenue is driven more by random/operational factors than by a predictable seasonal trend.

6. Cumulative Revenue Grows Steadily Despite Monthly Fluctuation

Even though monthly revenue swings sharply up and down, the cumulative revenue trend (Query 15) shows steady, consistent growth across all 24 months, confirming the business keeps growing in aggregate even when month-to-month performance is uneven.

7. Sales Are Evenly Spread Across the Week

No single day dominates extremely — the highest revenue day (Monday, 38,021.54) and the lowest (Saturday, 34,456.04) differ by only about 10%. This suggests customer shopping patterns aren't strongly influenced by the day of the week.

8. AOV Is Relatively Low — an Opportunity for Bundling/Cross-Sell

Average Order Value is only 78.33 across 3,220 Completed orders, much lower than the average revenue per customer (288.59) — meaning many customers return repeatedly but with small transaction values per visit. There's an opportunity to raise AOV through bundling or cross-selling complementary products.

5. Day 3 Checklist Summary

The table below shows the completion status of each Day 3 stage, taken from the portfolio progress tracker.

No	Day	Stage	Task	Tools	Status
28	Day 3	SQL	Import dataset into MySQL	MySQL	DONE
29	Day 3	SQL	Query: Total revenue	MySQL	DONE
30	Day 3	SQL	Query: Total orders	MySQL	DONE
31	Day 3	SQL	Query: Average order value	MySQL	DONE
32	Day 3	SQL	Query: Top products	MySQL	DONE
33	Day 3	SQL	Query: Top categories	MySQL	DONE
34	Day 3	SQL	Query: Monthly sales	MySQL	DONE
35	Day 3	SQL	Query: Regional sales	MySQL	DONE
36	Day 3	SQL	Query: Repeat customers	MySQL	DONE
37	Day 3	SQL	Query: Highest-value customers	MySQL	DONE
38	Day 3	SQL	Query: Lowest-performing products	MySQL	DONE
39	Day 3	SQL	Query: Growth MoM	MySQL	DONE
40	Day 3	SQL	Query: Category contribution	MySQL	DONE
41	Day 3	SQL	Query: Average revenue per customer	MySQL	DONE
42	Day 3	SQL	Query: Sales by weekday	MySQL	DONE
43	Day 3	SQL	Query: Sales trend	MySQL	DONE
44	Day 3	SQL	Minimum of 15 queries collected	MySQL	DONE
45	Day 3	SQL	Saved as business_analysis.sql	MySQL	DONE

Table 2. Status of all Day 3 SQL Analysis tasks

6. Conclusion

The Day 3 analysis produced `business_analysis.sql`, containing 15 documented business queries — from basic metrics (total revenue, total orders, AOV) to advanced analysis using window functions (month-over-month growth, cumulative revenue trend). The entire process was built on the clean 3,933-row dataset from Day 1, and every query result is mutually consistent (for example, `total_revenue` in Query 1 matches exactly the final `cumulative_revenue` in Query 15).

The most significant findings: extreme dependence on the East region (74.4% of revenue), Electronics as the primary sales driver, and an AOV that is relatively low compared to average revenue per customer — these three points complement and reinforce the insights already found in Day 2, and are strong candidates for deeper discussion in Day 6 (Business Case Study).

This `business_analysis.sql` file will serve as a cross-check/validation reference for Day 4 (Python EDA) and Day 5 (Tableau dashboard) — the KPI figures and insights above will be used to ensure consistency of results across every stage.